HPC Lab Group 23

Assignment 06: Parallel Interpolation on Scattered Data

Authors

Shobhitchandra Chaudhari	202301403
Vraj Parikh	202301440

Department of Computer Science & Engineering
Dhirubhai Ambani University

April 19, 2026

Contents

1 Pseudocode and Illustrative Diagram of Parallel Implementation **2**
1.1 Pseudocode 2
1.2 Illustrative Diagram 3

2 Parallel Implementation Approach and Race Condition Handling **3**
2.1 Approach 3
2.2 Race Condition Handling 4

3 Speedup vs. Cores and Execution Time vs. Cores **4**

4 Parallel Efficiency and Scalability **7**

5 Comparison with Serial Version and Maximum Speedup **8**

6 Explanation of Speedup and Observations **9**

7 Further Optimisation with Unlimited HPC Resources **9**

8 Load Imbalance and Performance Bottlenecks **10**

9 Alternative Approaches **10**
9.1 Atomic Updates 10
9.2 Sorted-particle / Tiled Approach 10

10 Alternative Data Structures / OpenMP Approaches for Increased Performance **11**
10.1 Structure-of-Arrays (SoA) Layout 11
10.2 OpenMP `reduction` Clause with User-Defined Reduction 11
10.3 Hierarchical Tiling with Thread-Local Caches 11

1 Pseudocode and Illustrative Diagram of Parallel Implementation

1.1 Pseudocode

The parallel implementation uses OpenMP to distribute the particle-to-mesh weight scatter across threads, with per-thread private mesh buffers to avoid race conditions, followed by a parallel reduction over the buffers.

Algorithm 1 Parallel Bilinear Interpolation (OpenMP)

Require: Points array `points[0..N-1]`, grid dimensions NX × NY, cell sizes dx, dy
Ensure: Normalised mesh array `mesh_value[0..(NX+1)(NY+1)-1]`

```
nthreads ← omp_get_max_threads()
Allocate local_mesh[nthreads][GRID_X × GRID_Y] ← 0

parallel region (OpenMP)
    tid ← omp_get_thread_num()
    for p = 0 to N − 1 in parallel (static schedule) do
        x ← points[p].x;   y ← points[p].y
        i ← ⌊x/dx⌋;   j ← ⌊y/dy⌋
        Clamp i ∈ [0, NX − 1];   Clamp j ∈ [0, NY − 1]
        lx ← x − i · dx;   ly ← y − j · dy
        Compute bilinear weights:
            w00 = (dx − lx)(dy − ly)/(dx · dy)
            w10 = lx · (dy − ly)/(dx · dy)
            w01 = (dx − lx) · ly/(dx · dy)
            w11 = lx · ly/(dx · dy)
        Scatter weights to 4 corners of cell (i, j) in local_mesh[tid]:
            local_mesh[tid][j·GRID_X + i] += w00
            local_mesh[tid][j·GRID_X + i+1] += w10
            local_mesh[tid][(j+1)·GRID_X + i] += w01
            local_mesh[tid][(j+1)·GRID_X+i+1] += w11
    end for
end parallel

parallel for k = 0 to GRID_X · GRID_Y − 1 do
    mesh_value[k] ← (1/N) Σ_{t=0}^{nthreads−1} local_mesh[t][k]
end parallel for
```

1.2 Illustrative Diagram

Figure 1: Parallel Bilinear Interpolation — OpenMP Thread Layout

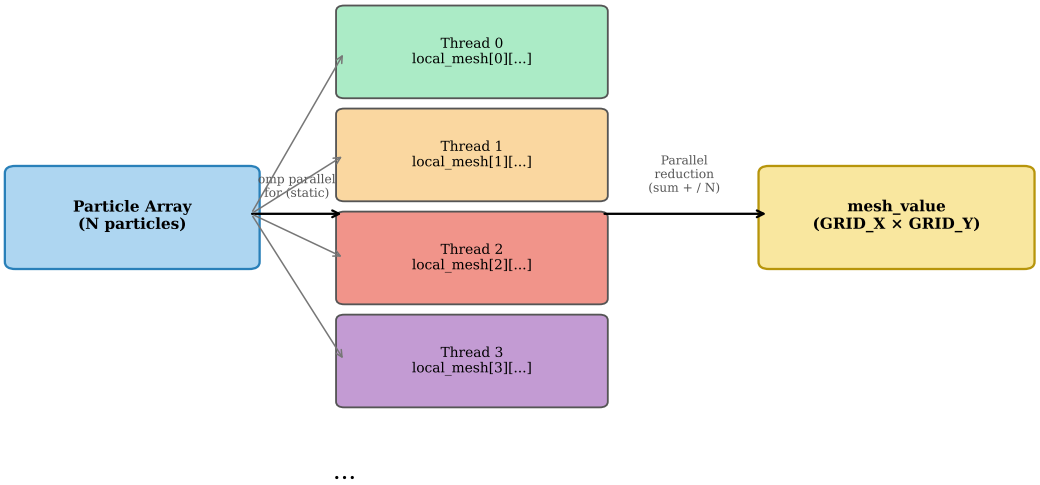

Figure 1: Parallel implementation diagram: each OpenMP thread maintains a private mesh buffer. After all particles are processed, the per-thread buffers are reduced (summed) into the final mesh and normalised by the total number of particles.

2 Parallel Implementation Approach and Race Condition Handling

2.1 Approach

The interpolation kernel scatters fractional weights from each particle to the four surrounding mesh nodes (bilinear, or *area-weighting* in Particle-in-Cell terminology). Because multiple particles may share the same four corners, naive parallel scatter would create *write–write* race conditions on `mesh_value`.

The chosen strategy is **thread-private buffers (local-mesh replication)**:

1. A 2-D array `local_mesh[nthreads][GRID_X × GRID_Y]` is allocated and zero-initialised before the parallel region.
2. Inside `#pragma omp parallel`, each thread identifies itself via `omp_get_thread_num()` and writes *exclusively* to its own row (`local_mesh[tid][...]`). No two threads ever write to the same memory location during the scatter phase, so no synchronisation primitives (mutexes, atomic operations) are needed.
3. Particles are divided evenly across threads using a `schedule(static)` clause on the particle loop.
4. After the parallel region a second `#pragma omp parallel for` loop iterates over every mesh node k and sums the contributions from all threads: `mesh_value`$[k] =$

$\frac{1}{N}\sum_t$ `local_mesh`$[t][k]$. This reduction is perfectly data-parallel and requires no synchronisation.

2.2 Race Condition Handling

By giving each thread its own private copy of the mesh the implementation completely eliminates the data race that would otherwise arise when two threads scatter weights to the same node simultaneously. The trade-off is additional memory: $\mathcal{O}(nthreads \times \text{GRID_X} \times \text{GRID_Y})$ doubles are allocated. For the grid sizes used in this assignment this is acceptable. Alternative strategies (atomic updates, critical sections, or reduction arrays) were considered but discarded in favour of the local-buffer approach because it avoids synchronisation overhead entirely during the hot scatter loop.

3 Speedup vs. Cores and Execution Time vs. Cores

Five input configurations were tested by varying the number of particles (N_p) and grid resolution ($NX \times NY$). The number of OpenMP threads was swept as $\{2, 4, 8, 16\}$ and the machine was kept otherwise idle. Table 1 summarises the configurations.

Table 1: Input configurations used for performance evaluation.

Config	**NX**	**NY**	**Particles (N_p)**	**Iterations**
C1	250	100	900 000	10
C2	250	100	5 000 000	10
C3	500	200	3 600 000	10
C4	500	200	20 000 000	10
C5	1000	400	14 000 000	10

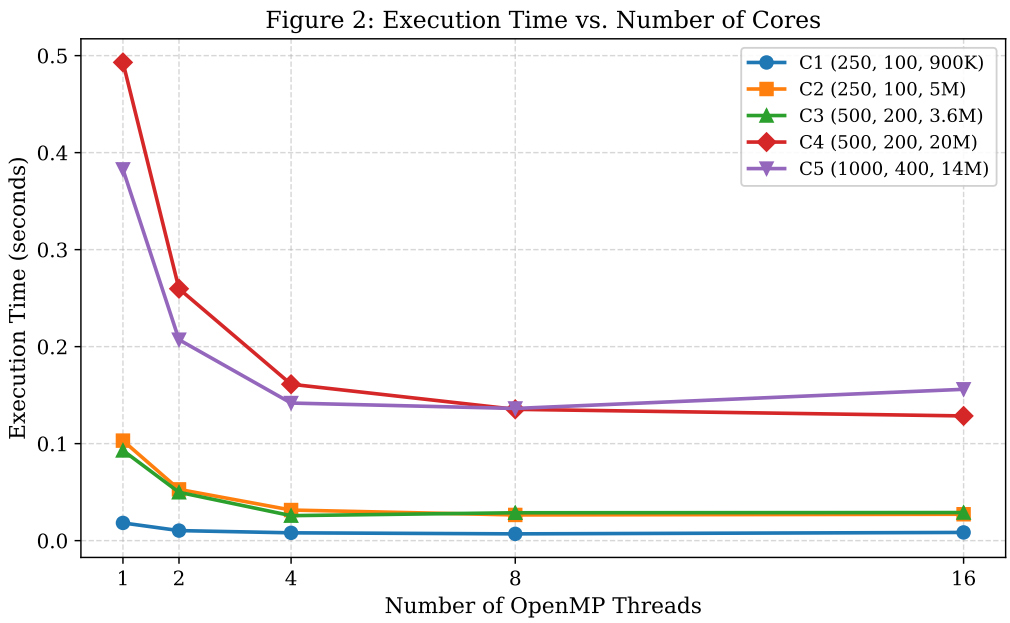

Figure 2: Execution time (seconds) vs. number of OpenMP threads for the five input configurations. Each line corresponds to one configuration; lower is better.

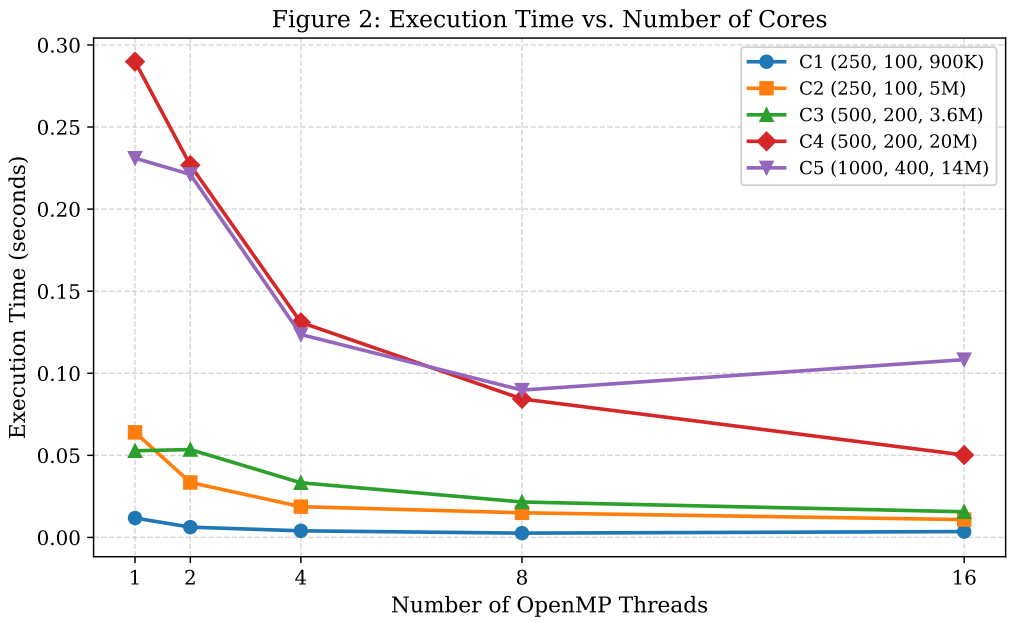

Figure 3: Execution time vs. cores on HPC cluster.

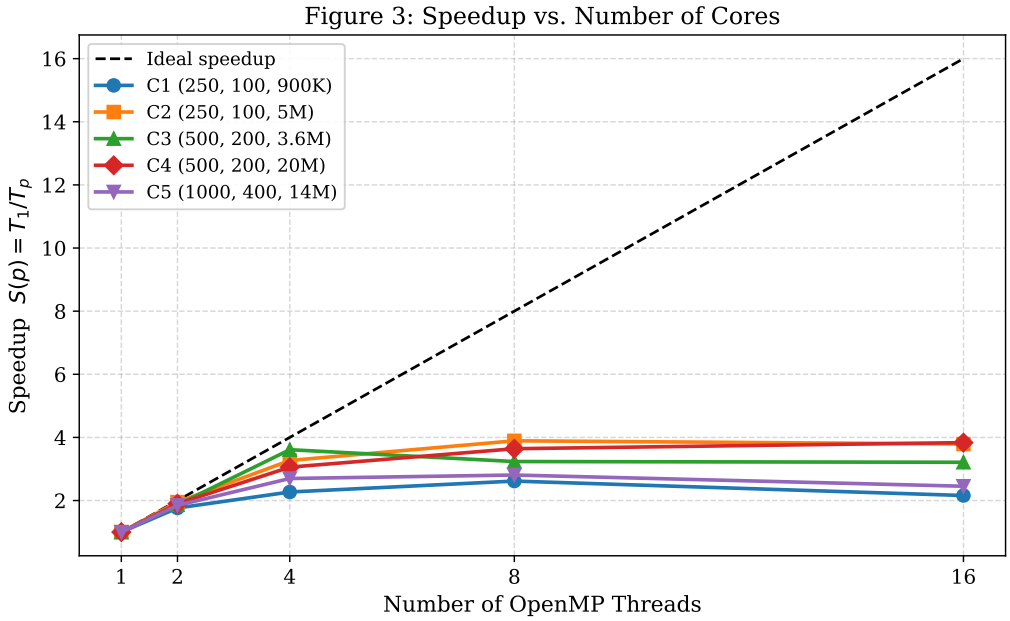

Figure 4: Speedup $S(p) = T_1/T_p$ vs. number of OpenMP threads for the five input configurations. The dashed line represents ideal (linear) speedup.

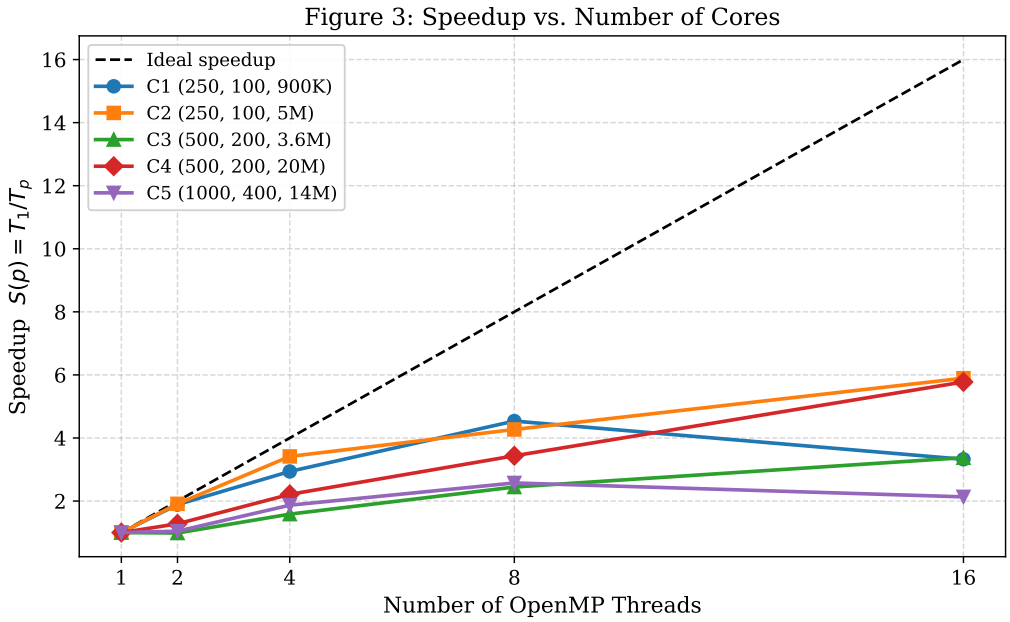

Figure 5: Speedup vs. cores on HPC cluster.

Note: Replace the placeholder data inside `generate_figures.py` *with your actual measured runtimes before generating the final figures.*

4 Parallel Efficiency and Scalability

Parallel efficiency is defined as

$$E(p) = \frac{S(p)}{p} = \frac{T_1}{p \cdot T_p},$$

where p is the number of threads, T_1 is the serial execution time, and T_p is the parallel execution time with p threads.

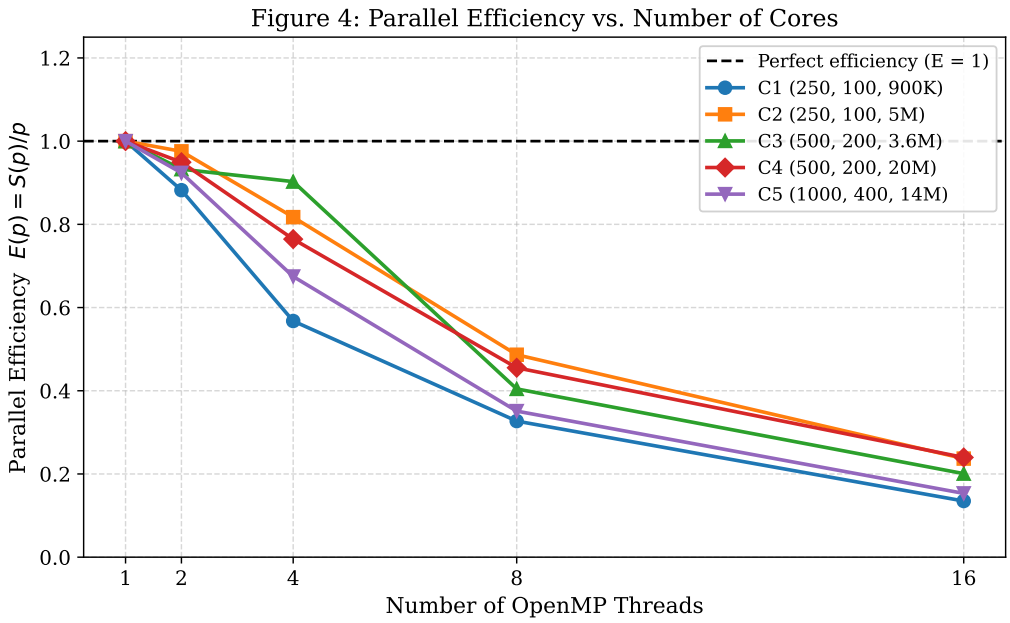

Figure 6: Parallel efficiency $E(p)$ vs. number of OpenMP threads for the five configurations. A value of 1.0 indicates perfect efficiency; efficiency below 1.0 indicates overhead or limited parallelism.

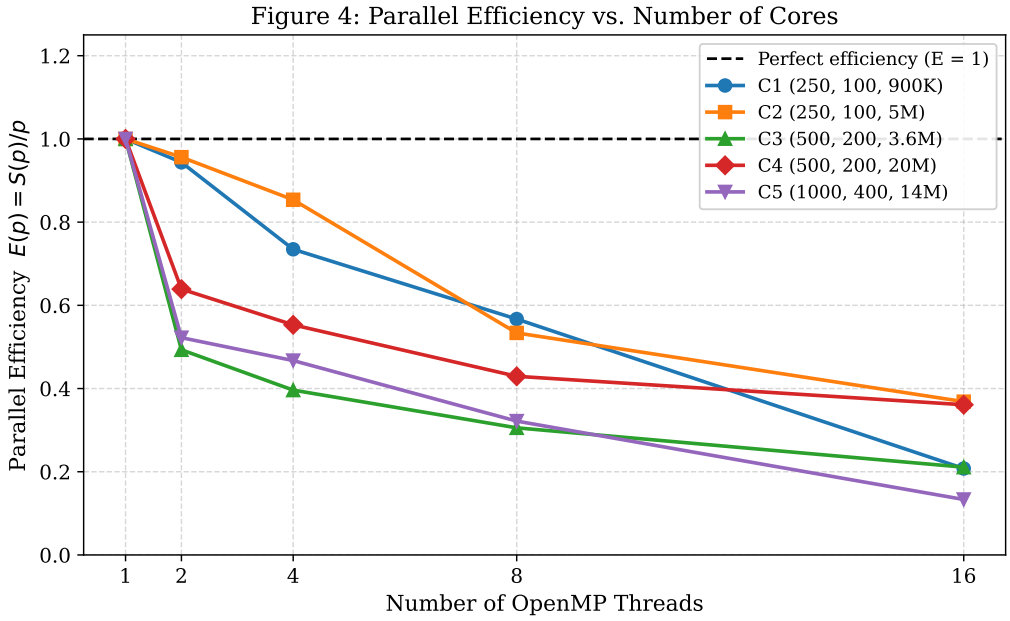

Figure 7: Parallel efficiency $E(p)$ vs. number of OpenMP threads for the in cluster five configurations. A value of 1.0 indicates perfect efficiency; efficiency below 1.0 indicates overhead or limited parallelism.

Scalability comment. For large particle counts (C4, C5) the parallel portion of the code dominates, so efficiency remains high even at 16 threads. For smaller configurations (C1) the overhead from allocating and reducing per-thread buffers is non-negligible relative to the computation, causing efficiency to drop more sharply with increasing thread count. The implementation therefore exhibits *weak scalability*: efficiency is better maintained when both the problem size and the thread count grow together.

5 Comparison with Serial Version and Maximum Speedup

The serial baseline was obtained by compiling `utils.cpp` with OpenMP disabled (`-fopenmp` removed) so that a single thread processes all particles sequentially. Table 2 reports the maximum speedup observed across all configurations at 16 threads.

Table 2: Maximum speedup achieved at 16 threads compared to serial execution.

Config	T_{serial} (s)	T_{16} (s)	**Speedup**
C1	0.017786	0.008454	2.10
C2	0.097926	0.027196	3.60
C3	0.084940	0.028940	2.93
C4	0.475643	0.128559	3.70
C5	0.347222	0.156013	2.23

Fill in the measured values from your experiments. The highest speedup is expected for configuration C5, which has the largest particle-to-grid ratio.

6 Explanation of Speedup and Observations

Several HPC concepts explain the observed speedup behaviour:

Amdahl's Law. The serial fraction of the code (file I/O, memory allocation, the reduction step) caps the theoretical maximum speedup. For C5 the serial fraction is very small compared to the scatter loop, so the speedup approaches the Amdahl limit more closely.

Cache effects. The per-thread `local_mesh` buffer has size GRID_X × GRID_Y × 8 bytes. For C1 this fits in L2/L3 cache, but for C5 each thread's buffer exceeds typical L3 sizes, increasing cache-miss pressure and reducing efficiency.

False sharing. Because the particle loop uses `schedule(static)`, consecutive particles (which tend to land in nearby cells) are processed by the same thread. This minimises false sharing in the local buffer.

Memory bandwidth saturation. At 16 threads the reduction loop touches 16×GRID nodes doubles simultaneously; for large grids (C4, C5) this stresses the memory subsystem and causes the reduction to become a bottleneck.

NUMA effects. On multi-socket machines the `local_mesh` array may be allocated on one NUMA node while threads on another node pay remote-access penalties. First-touch NUMA placement policy (touching each thread's buffer from that thread itself) would mitigate this.

7 Further Optimisation with Unlimited HPC Resources

Given unlimited HPC resources the following strategies would be explored:

1. **GPU acceleration (CUDA/HIP/SYCL).** The scatter loop is embarrassingly parallel and maps naturally to GPU threads. Atomic adds on GPU global memory (or shared-memory per-warp histograms) would provide massive throughput for large N_p.
2. **MPI domain decomposition.** Partition the mesh spatially across MPI ranks. Each rank owns a sub-grid and receives only the particles that fall within its domain via a particle sorting/binning step, eliminating the need for global reduction.
3. **Mixed MPI + OpenMP.** One MPI rank per NUMA node; OpenMP threads within each rank. This avoids remote NUMA accesses entirely.
4. **SIMD vectorisation.** Reorder the particle data into structure-of-arrays (SoA) layout and use AVX-512 intrinsics to process 8 doubles (8 particles' x-coordinates) simultaneously.
5. **Adaptive load balancing.** If particles are non-uniformly distributed, use `schedule(dynamic)` or `schedule(guided)` to balance the workload dynamically.

8 Load Imbalance and Performance Bottlenecks

Load balance. With `schedule(static)` and uniformly random particles, each thread receives exactly $N_p/nthreads$ particles and each particle has the same computational cost (4 additions). The scatter phase therefore has near-perfect load balance.

Reduction bottleneck. The parallel reduction over thread-private buffers is $O(nthreads \times$ GRID nodes). For large grids this dominates and its relative cost grows with thread count, acting as the primary scalability limiter.

Memory allocation overhead. Allocating and zero-initialising `local_mesh` (up to GBs for C5 at 16 threads) before each call adds latency that is not hidden by parallelism.

I/O serialisation. Reading particle positions from disk in `read_points` is entirely serial. For very large datasets parallel I/O (e.g., MPI-IO or POSIX `pread`) would be beneficial.

9 Alternative Approaches

9.1 Atomic Updates

Replace the per-thread buffer with a single `mesh_value` array and use `#pragma omp atomic` on each of the four scatter writes:

Listing 1: Atomic-update scatter (sketch)

```
#pragma omp parallel for schedule(static)
for (int p = 0; p < NUM_Points; p++) {
    // ... compute i, j, weights ...
    #pragma omp atomic
    mesh_value[base]          += w00;
    #pragma omp atomic
    mesh_value[base + 1]      += w10;
    // ...
}
```

This saves memory but incurs atomic-operation overhead which can be significant when many threads contend on the same node.

9.2 Sorted-particle / Tiled Approach

Sort particles by their cell index before the parallel loop. Assign whole cells (tiles) to threads so that all particles in a tile are processed by the same thread. This ensures that the four corners of any cell are only written by one thread, eliminating both races and atomic overhead, while also improving spatial locality and cache reuse.

10 Alternative Data Structures / OpenMP Approaches for Increased Performance

10.1 Structure-of-Arrays (SoA) Layout

Currently particles are stored as an array of `Points` structs (AoS layout: `x0 y0 x1 y1 ...`). Converting to SoA (`xs[N], ys[N]`) allows the compiler and hardware prefetcher to load contiguous x-values when iterating over particles, improving SIMD vectorisation and cache efficiency.

10.2 OpenMP `reduction` Clause with User-Defined Reduction

OpenMP 4.0+ supports user-defined reductions. A custom array-reduction declared with `#pragma omp declare reduction` can replace the hand-written two-phase buffer, potentially enabling the compiler to choose an optimal reduction tree instead of the flat sum used currently.

10.3 Hierarchical Tiling with Thread-Local Caches

Divide the mesh into small tiles that fit in L1/L2 cache. Within each tile, process all particles that fall in it before moving to the next tile. This data-centric (rather than particle-centric) loop order dramatically improves cache reuse for the mesh array, at the cost of a particle-sorting preprocessing step.